Risk Score Weighted Calculation

Difficulty: Intermediate | Topic: Banking Logic and Risk Evaluation | Language: COBOL

1. Problem Statement

Calculate a weighted risk score from three Boolean indicators and their corresponding point values. Use two configured thresholds to classify the score as LOW, MEDIUM, or HIGH. Reject invalid input or an invalid threshold configuration.

2. Input Format

Enter eight space-separated integer values on one line in this order:

```
indicator1 indicator2 indicator3 points1 points2 points3 lowThreshold highThreshold
```

Each indicator must be 0 or 1. Point values and thresholds must be non-negative, and highThreshold must be greater than or equal to lowThreshold.

3. Sample Input

All three indicators are active, giving a score of 5 + 3 + 2 = 10. With thresholds 4 and 8, the score is classified as HIGH.

```
1 1 1 5 3 2 4 8
```

4. Sample Output

```
SCORE: 10
RISK BAND: HIGH
```

5. COBOL Code

```
       >>SOURCE FORMAT FREE
IDENTIFICATION DIVISION.
PROGRAM-ID. RISK-SCORE.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 INPUT-LINE             PIC X(200).
01 TOKEN-COUNT            PIC 9(2) COMP VALUE 0.
01 TOKENS.
   05 TOKEN               PIC X(20) OCCURS 8 TIMES.
01 INDICATOR-1            PIC S9(9) COMP-5 VALUE 0.
01 INDICATOR-2            PIC S9(9) COMP-5 VALUE 0.
01 INDICATOR-3            PIC S9(9) COMP-5 VALUE 0.
01 POINTS-1               PIC S9(9) COMP-5 VALUE 0.
01 POINTS-2               PIC S9(9) COMP-5 VALUE 0.
01 POINTS-3               PIC S9(9) COMP-5 VALUE 0.
01 LOW-THRESHOLD          PIC S9(9) COMP-5 VALUE 0.
01 HIGH-THRESHOLD         PIC S9(9) COMP-5 VALUE 0.
01 RISK-SCORE             PIC 9(10) COMP-5 VALUE 0.
01 DISPLAY-SCORE          PIC Z(9)9.
01 VALID-INPUT            PIC X VALUE 'Y'.
```

```
01 TOKEN-INDEX           PIC 9(2) COMP VALUE 0.

PROCEDURE DIVISION.
    ACCEPT INPUT-LINE
    UNSTRING INPUT-LINE DELIMITED BY ALL SPACES
        INTO TOKEN(1) TOKEN(2) TOKEN(3) TOKEN(4)
             TOKEN(5) TOKEN(6) TOKEN(7) TOKEN(8)
        TALLYING IN TOKEN-COUNT
    END-UNSTRING

    IF TOKEN-COUNT NOT = 8
        MOVE 'N' TO VALID-INPUT
    ELSE
        PERFORM VARYING TOKEN-INDEX FROM 1 BY 1
            UNTIL TOKEN-INDEX > 8
            IF FUNCTION TEST-NUMVAL(
                FUNCTION TRIM(TOKEN(TOKEN-INDEX))) NOT = 0
                MOVE 'N' TO VALID-INPUT
            END-IF
        END-PERFORM
    END-IF

    IF VALID-INPUT = 'Y'
        COMPUTE INDICATOR-1 = FUNCTION NUMVAL(
            FUNCTION TRIM(TOKEN(1)))
        COMPUTE INDICATOR-2 = FUNCTION NUMVAL(
            FUNCTION TRIM(TOKEN(2)))
        COMPUTE INDICATOR-3 = FUNCTION NUMVAL(
            FUNCTION TRIM(TOKEN(3)))
        COMPUTE POINTS-1 = FUNCTION NUMVAL(
            FUNCTION TRIM(TOKEN(4)))
        COMPUTE POINTS-2 = FUNCTION NUMVAL(
            FUNCTION TRIM(TOKEN(5)))
        COMPUTE POINTS-3 = FUNCTION NUMVAL(
            FUNCTION TRIM(TOKEN(6)))
        COMPUTE LOW-THRESHOLD = FUNCTION NUMVAL(
            FUNCTION TRIM(TOKEN(7)))
        COMPUTE HIGH-THRESHOLD = FUNCTION NUMVAL(
            FUNCTION TRIM(TOKEN(8)))

        IF (INDICATOR-1 NOT = 0 AND INDICATOR-1 NOT = 1)
           OR (INDICATOR-2 NOT = 0 AND INDICATOR-2 NOT = 1)
           OR (INDICATOR-3 NOT = 0 AND INDICATOR-3 NOT = 1)
           OR POINTS-1 < 0
           OR POINTS-2 < 0
           OR POINTS-3 < 0
           OR LOW-THRESHOLD < 0
           OR HIGH-THRESHOLD < LOW-THRESHOLD
            MOVE 'N' TO VALID-INPUT
        END-IF
    END-IF

    IF VALID-INPUT NOT = 'Y'
        DISPLAY 'INVALID CONFIGURATION'
    ELSE
        COMPUTE RISK-SCORE =
            (INDICATOR-1 * POINTS-1)
          + (INDICATOR-2 * POINTS-2)
          + (INDICATOR-3 * POINTS-3)
        MOVE RISK-SCORE TO DISPLAY-SCORE
        DISPLAY 'SCORE: ' FUNCTION TRIM(DISPLAY-SCORE)
```

```
        EVALUATE TRUE
            WHEN RISK-SCORE <= LOW-THRESHOLD
                DISPLAY 'RISK BAND: LOW'
            WHEN RISK-SCORE <= HIGH-THRESHOLD
                DISPLAY 'RISK BAND: MEDIUM'
            WHEN OTHER
                DISPLAY 'RISK BAND: HIGH'
        END-EVALUATE
    END-IF

    STOP RUN.
```

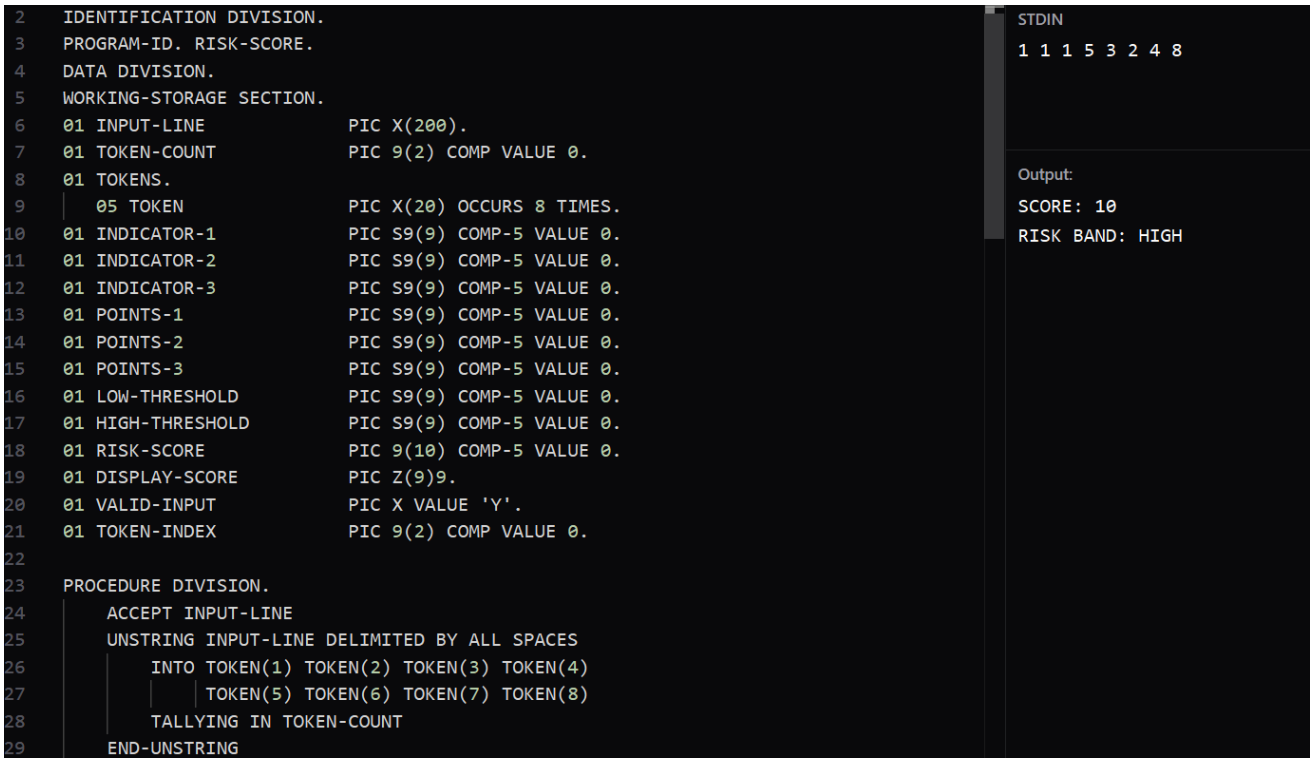


Fig. Program Execution and Output